

Udemy Analytics Pipeline

Lunes 14 → presentación lunes 21 · ETL en Python + frontend React sobre 573 cursos reales de Udemy

Contexto y alcance de esta ronda

El punto de partida es un export real de Udemy (`udemy_biblioteca_completa.xlsx`) con 5 hojas: **Resumen** (573 cursos, progreso, duración, tecnologías), **Temario Completo**, **Objetivos de Aprendizaje**, **Cursos No Disponibles** y **Exámenes de Práctica**. Es un dataset real, con nulos, texto libre y formatos inconsistentes.

Para caber en una semana, esta ronda es **ETL en Python** → `courses.json` limpio → **frontend React** que lo consume directamente. Nada de base de datos ni API todavía — eso queda para una fase 2 futura. Existe un dashboard de referencia (HTML+Tailwind+Chart.js) que muestra hasta dónde se puede llegar; no es para copiar, es para inspirarse en la arquitectura de features.

Reparto de roles (equipo trabajando en paralelo desde el día 1)

La clave para que 1 semana alcance: **todos arrancan el lunes en paralelo**, no en cadena. Esto solo funciona si a primera hora del lunes el equipo completo acuerda el *contrato de datos* — el shape exacto de `courses.json` — para que frontend no espere a que el ETL termine.

Rol	Personas	Entregable de la semana
Datos / ETL	1–2	Script Python que limpia el Excel y exporta <code>courses.json</code> , + tests con pytest
Frontend — Fundaciones	1	Scaffold Vite+React+TS, schema Zod, store Zustand, sync con nuqs
Frontend — Catálogo	1–2	Tabla con TanStack Table + panel de detalle con el temario
Frontend — Visualización	1–2	Gráficos con Chart.js, feature con Temporal, animaciones con Motion

Equipo más chico (3 personas): fusionar Catálogo + Visualización en una sola persona. Equipo más grande (6+): separar Chart.js/Temporal de Motion como roles distintos. Lo que no cambia es el rol de Datos/ETL — necesita autonomía propia desde el día 1.

Ritual diario: stand-up de 10 min cada mañana. Sirve sobre todo para que Datos/ETL avise de inmediato si el shape de `courses.json` cambia — es la dependencia que puede bloquear a los demás roles.

Cronograma día a día

Día	Datos / ETL	Frontend
Lun 14	Acordar el contrato del JSON + leer las 5 hojas del Excel	Scaffold del proyecto, instalar las 7 libs, schema Zod a partir del contrato acordado
Mar 15	Limpiar duraciones/tecnologías/estudiantes, categorizar, exportar <code>courses.json</code> + 2-3 tests pytest	Cargar y validar el JSON con Zod; store de filtros en Zustand; sync con nuqs en la URL
Mié 16	Buffer / ajustes al JSON si el frontend detecta huecos	Catálogo con TanStack Table (sort, filtros, paginación) + drawer de detalle con el temario
Jue 17	—	Gráficos con Chart.js (tecnologías, progreso) + feature con Temporal: dado un ritmo de estudio (h/semana), calcular fecha estimada de término
Vie 18	Todo el equipo: animaciones con Motion (tabs, drawer, filas) + estados de carga/vacío/error + integración final	
Sáb-Dom	Buffer para imprevistos, ensayo de la demo, README	
Lun 21	Presentación (10 min): cada rol muestra su parte + 2-3 decisiones técnicas + qué queda para una fase 2 (API+DB, recomendador)	

Must-have (para aprobar)

- ETL en Python reproducible: un comando reconstruye `courses.json` desde el Excel
- Mínimo 2 tests reales con `pytest` (parser de duración, split de tecnologías)
- Catálogo con TanStack Table + filtros vía Zustand, reflejados en la URL con nuqs
- Validación del dataset con Zod al cargarlo
- Al menos 2 gráficos con Chart.js
- 1 feature real usando Temporal (no solo "está importado")
- Estados de carga/vacío/error en la vista principal

Nice-to-have (si sobra tiempo)

- Motion en más de un lugar de la interfaz
- Sistema de notas por curso (aunque sea en localStorage, sin backend)
- Mención en la demo de cómo pasarían esto a FastAPI+DB en una fase 2